



# NDI Bridge Service

2025.03.18

NDI© 2024

---

# Contents

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                       | <b>3</b>  |
| <b>2</b> | <b>Installation</b>                       | <b>4</b>  |
| 2.1      | Silent Installation . . . . .             | 7         |
| <b>3</b> | <b>Configuration</b>                      | <b>8</b>  |
| 3.1      | Web UI . . . . .                          | 8         |
| 3.1.1    | Status Information . . . . .              | 8         |
| 3.1.2    | API Keys . . . . .                        | 9         |
| 3.2      | Configuration File . . . . .              | 11        |
| 3.3      | Manual API Key Management . . . . .       | 13        |
| <b>4</b> | <b>Webhooks</b>                           | <b>14</b> |
| <b>5</b> | <b>WebSockets</b>                         | <b>15</b> |
| <b>6</b> | <b>API</b>                                | <b>17</b> |
| 6.1      | API Calls Using SSL . . . . .             | 18        |
| 6.2      | Auto Start . . . . .                      | 19        |
| 6.3      | GPU List . . . . .                        | 19        |
| 6.4      | Host Bandwidth . . . . .                  | 19        |
| 6.5      | Host Bridge Name . . . . .                | 19        |
| 6.6      | Host Buffer . . . . .                     | 20        |
| 6.7      | Host Clear Encryption Key . . . . .       | 20        |
| 6.8      | Host GPU Limit . . . . .                  | 20        |
| 6.9      | Host Groups . . . . .                     | 21        |
| 6.10     | Host HX Encoder . . . . .                 | 21        |
| 6.11     | Host HX GPU . . . . .                     | 21        |
| 6.12     | Host HX NDI4 Compatibility Mode . . . . . | 22        |
| 6.13     | Host HX Output . . . . .                  | 22        |
| 6.14     | Host HX Quality . . . . .                 | 22        |
| 6.15     | Host Port . . . . .                       | 23        |
| 6.16     | Host Set Encryption Key . . . . .         | 23        |
| 6.17     | Host Start . . . . .                      | 23        |
| 6.18     | Host Status . . . . .                     | 24        |
| 6.19     | Host Stop . . . . .                       | 24        |
| 6.20     | Is Running . . . . .                      | 24        |
| 6.21     | Join Bandwidth . . . . .                  | 24        |
| 6.22     | Join Bridge Name . . . . .                | 24        |
| 6.23     | Join Buffer . . . . .                     | 25        |
| 6.24     | Join Clear Encryption Key . . . . .       | 25        |
| 6.25     | Join GPU Limit . . . . .                  | 25        |
| 6.26     | Join Groups . . . . .                     | 25        |
| 6.27     | Join HX Encoder . . . . .                 | 26        |
| 6.28     | Join HX GPU . . . . .                     | 26        |

---

|      |                                            |    |
|------|--------------------------------------------|----|
| 6.29 | Join HX NDI4 Compatibility Mode . . . . .  | 26 |
| 6.30 | Join HX Output . . . . .                   | 27 |
| 6.31 | Join HX Quality . . . . .                  | 27 |
| 6.32 | Join IP Address . . . . .                  | 28 |
| 6.33 | Join Port . . . . .                        | 28 |
| 6.34 | Join Set Encryption Key . . . . .          | 28 |
| 6.35 | Join Start . . . . .                       | 29 |
| 6.36 | Join Status . . . . .                      | 29 |
| 6.37 | Join Stop . . . . .                        | 29 |
| 6.38 | Local Bridge Name . . . . .                | 29 |
| 6.39 | Local Groups . . . . .                     | 30 |
| 6.40 | Local HX GPU . . . . .                     | 30 |
| 6.41 | Local HX NDI4 Compatibility Mode . . . . . | 30 |
| 6.42 | Local HX Output . . . . .                  | 31 |
| 6.43 | Local HX Quality . . . . .                 | 31 |
| 6.44 | Local Send Groups . . . . .                | 31 |
| 6.45 | Local Start . . . . .                      | 32 |
| 6.46 | Local Status . . . . .                     | 32 |
| 6.47 | Local Stop . . . . .                       | 32 |
| 6.48 | Connection Test Start . . . . .            | 32 |
| 6.49 | Connection Test Stop . . . . .             | 33 |
| 6.50 | Connection Test Bitrate . . . . .          | 33 |
| 6.51 | Run Mode . . . . .                         | 33 |
| 6.52 | Uptime . . . . .                           | 33 |

---

# 1 Introduction

NDI Bridge Service enables you to run NDI Bridge headless as a Windows Service. It provides an API for configuring and controlling NDI Bridge in much the same way as the desktop application variant does. Additionally, NDI Bridge Service includes an optional web interface with API documentation and other service configuration tools. With support for SSL, you can confidently expose NDI Bridge Service beyond your local network for remote control from anywhere in the world.

This document provides a comprehensive guide to installing, configuring, and using the NDI Bridge Service. From initial setup to advanced configuration and API use, the following sections will equip users with the knowledge needed to effectively utilize the NDI Bridge Service in their video production workflows.

---

## 2 Installation

NDI Bridge Service uses a standard installation wizard. Follow the prompts to install and configure NDI Bridge Service.

*NOTE:* If you plan to use SSL, please ensure your SSL certificate is installed in the local machine certificate store under the logical store path 'Personal/Certificates', as this is the only location where NDI Bridge Service looks for your certificate with a matching CN subject. For more information on installing and configuring SSL, see the Configuration section of this document.

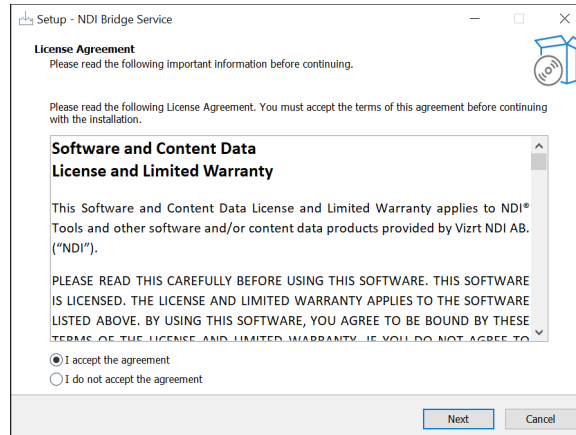


Figure 1: Installation wizard

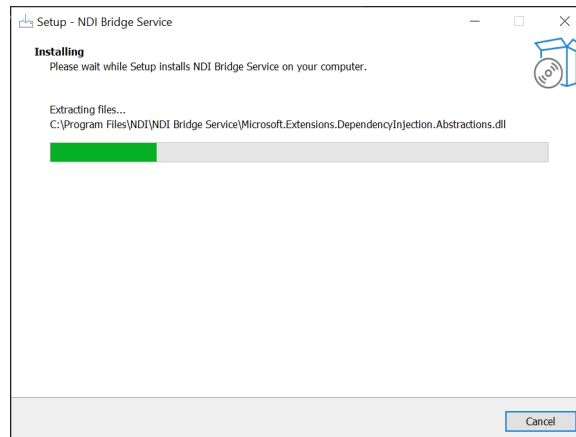


Figure 2: Installation wizard

Setup - NDI Bridge Service

**Service Configuration**  
Please configure the service settings below:

Port:

Host Binding:

☒ Enable Web Content

☒ Enable SSL

Certificate:

Certificates must be installed in the local machine  
certificate store and contain the "Server Authentication" OID.

Finish

Figure 3: Configuration wizard

At the end of the installation wizard, you will be presented with a configuration page to provide the initial configuration for NDI Bridge Service.

- *Port*: This is the port that the web server uses to provide access to the web interface and APIs. It is not the port used by Bridge in Host or Join mode; that port is configured via the API along with all other Bridge settings. By default, this value is 8080, but it can be changed to any valid port number. If you enable SSL and did not modify the port field from its default 8080, it will be changed to 443 by default.
- *Host Binding*: This is the interface that the web server listens on. By default, the installer sets this value to 127.0.0.1 (the local loopback device). You can change this to target any specific interface on the machine or use 0.0.0.0 to target all interfaces. If you enable SSL and did not modify the Host Binding field from its default of 127.0.0.1, it will be changed to 0.0.0.0 by default.
- *Enable Web Content*: Enabled by default, you can uncheck this field to disable the web

---

interface. Only the API will be functioning and there will be no accessible web page.

- *Enable SSL:* Check this field to enable SSL. Before finishing the installation you must also select a valid certificate from the drop down menu.
- *Certificate:* If SSL is enabled, you must select a valid certificate from the drop-down menu. SSL certificates must be installed in the local machine certificate store under the logical store path 'Personal/Certificates', as this is the only location where NDI Bridge Service looks for your certificate with a matching CN subject. Your certificate should be visible from the drop-down menu if you did this before running the installer. If you need to modify this or any other setting after installation, you can do so manually by following the instructions in the Configuration section of this document.

Once the installation is finished, the service will be started. If the 'Enable Web Content' field was checked on the configuration page, your browser will open to the NDI Bridge Service web page. Additionally, if you enabled SSL, you will be required to set up a password for the 'admin' account upon navigating to the web page. The password must contain upper and lowercase letters, numbers, and symbols.

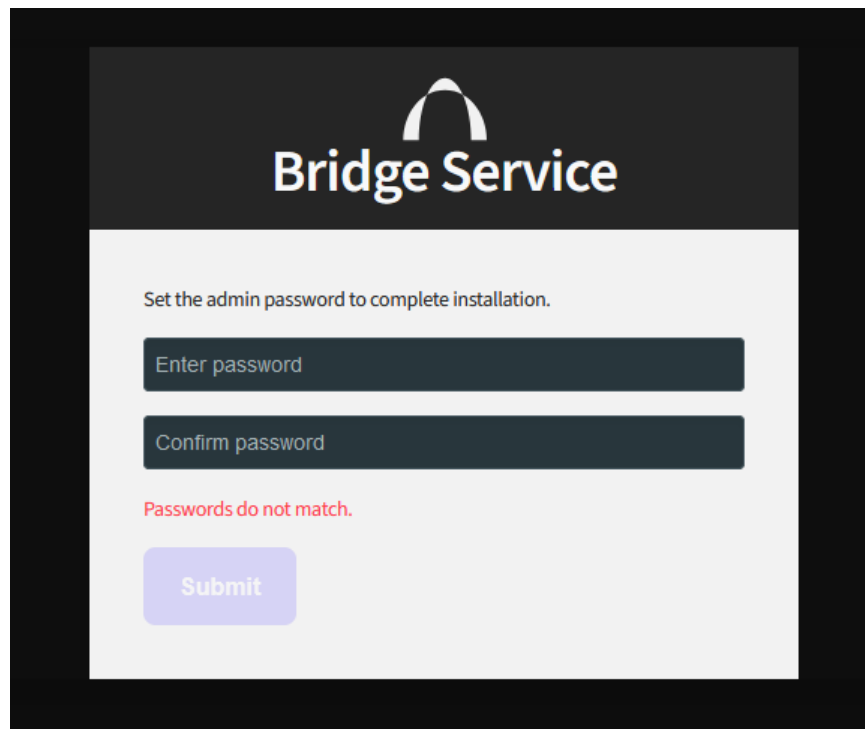
The image shows a web browser window displaying the NDI Bridge Service interface. At the top, there is a dark header with the NDI logo (a stylized arch) and the text "Bridge Service" in white. Below the header, the main content area has a light gray background. It contains the instruction "Set the admin password to complete installation." in a small, dark font. There are two dark gray input fields: the first is labeled "Enter password" and the second is labeled "Confirm password". Below these fields, a red error message reads "Passwords do not match." At the bottom of the form, there is a light purple button with the word "Submit" in white text.

Figure 4: Set admin password prompt

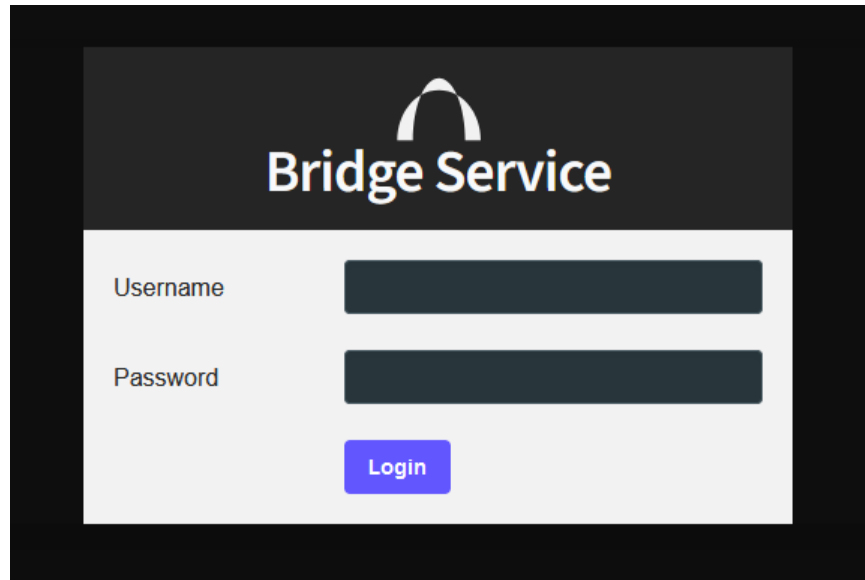


Figure 5: Login prompt

## 2.1 Silent Installation

The Bridge Service installer supports silent installation, allowing you to specify your desired configuration via the command line instead of using the installation wizard. The following command-line arguments are supported:

- */verysilent* This flag is required to perform a silent installation.
- */web* This flag specifies whether the web interface should run. It is optional and defaults to true. Valid options are true or false.
- */port* This flag specifies the port on which the service should run. It is not optional.
- */binding* This flag specifies which interface to bind to. It is not optional. Use 0.0.0.0 for broadcast.
- */ssl* This flag specifies whether or not to use SSL. It is optional and defaults to false. If this flag is set to true, the */cert* flag is also required. Valid options are true or false.
- */cert* This flag specifies the certificate to use when SSL is enabled. It is not optional when SSL is enabled. The certificate should be in Common Name format (CN=). SSL certificates must be installed in the local machine certificate store under the "Personal/Certificates" logical store path.

Example use: `/verysilent /web=true /ssl=true /port=443 /binding=0.0.0.0 /cert=mycertificate.local`

---

## 3 Configuration

The NDI Bridge Service web page provides an API reference, runtime status information, webhook management, WebSocket management, and API key management if SSL is enabled.

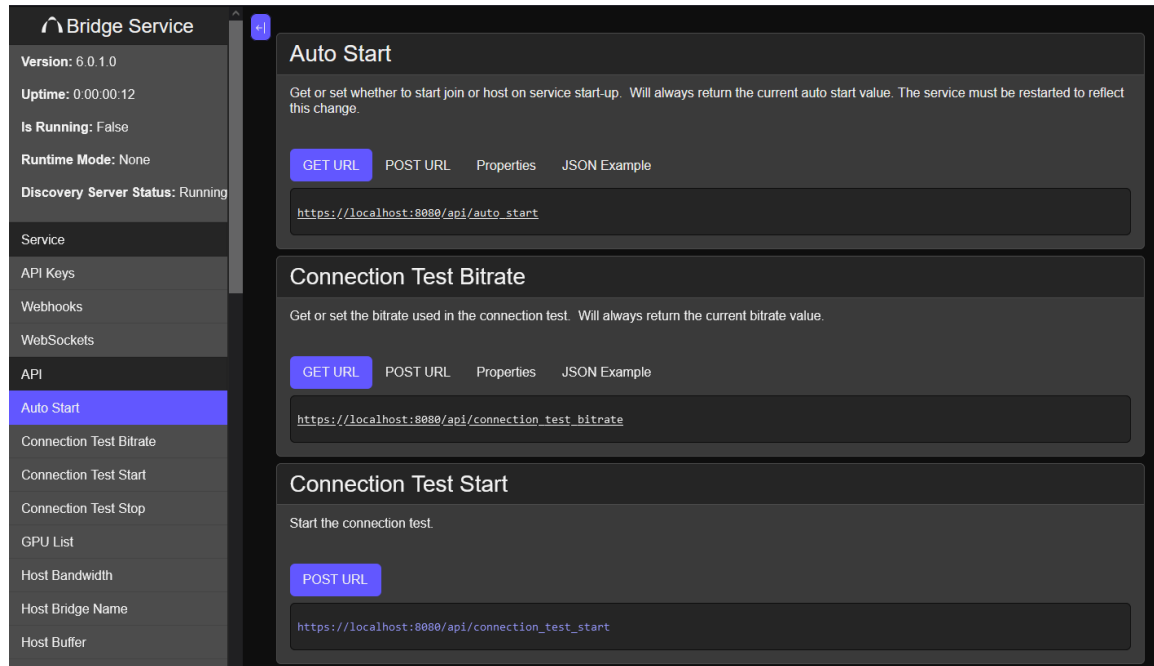


Figure 6: Main page

### 3.1 Web UI

#### 3.1.1 Status Information

The upper left-hand corner of the web page includes the status pane, which provides information about the current status of NDI Bridge Service.

- *Version*: This indicates the version of NDI Bridge Service that is installed and running.
- *Uptime*: This indicates the length of time NDI Bridge Service has been running. This is not the length of time Bridge is running. NDI Bridge Service can be running without Bridge itself running in Host, Join or Local mode.
- *Is Running*: This indicates whether Bridge is running in either Host or Join mode.
- *Runtime Mode*: This indicates which mode Bridge is running in: HOST for Host mode, JOIN for Join mode or Local for local mode.
- *Discovery Server Status*: This indicates that the NDI Discovery Server, a component required for Bridge to function properly, is running in the background.

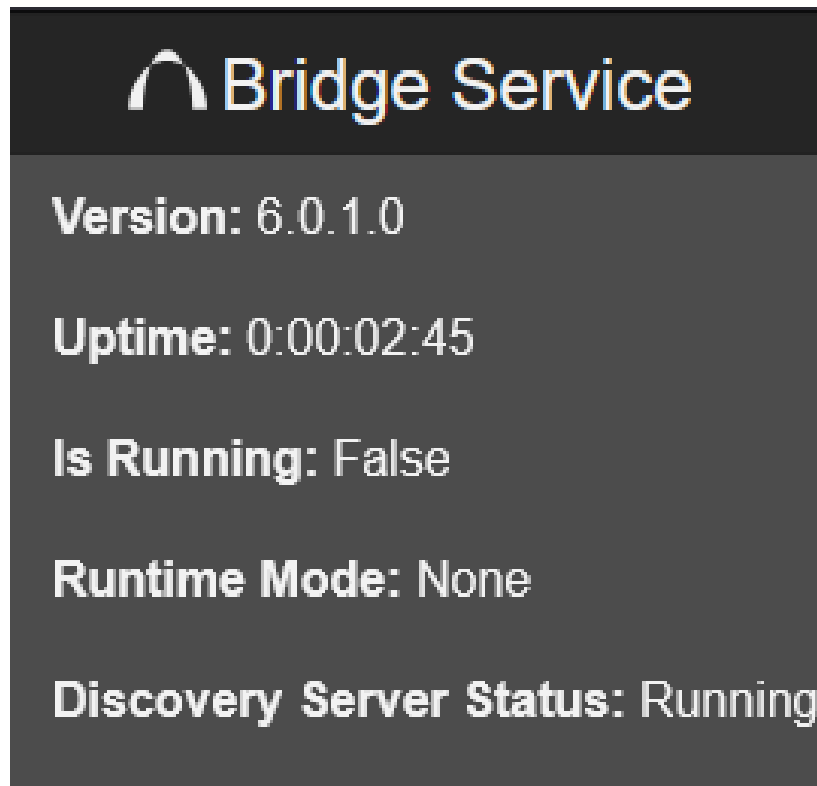


Figure 7: Status pane

### 3.1.2 API Keys

If SSL is enabled and properly configured, you will see an 'API Keys' section on the left-hand navigation menu. On this page, you can add and remove API keys required for accessing the API when SSL is enabled.

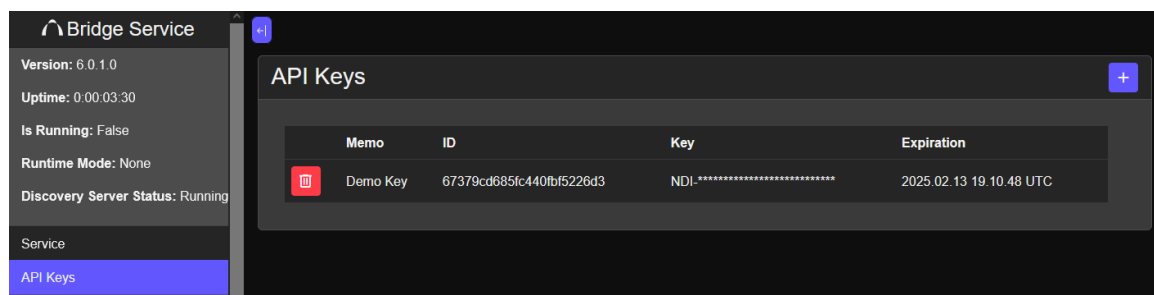
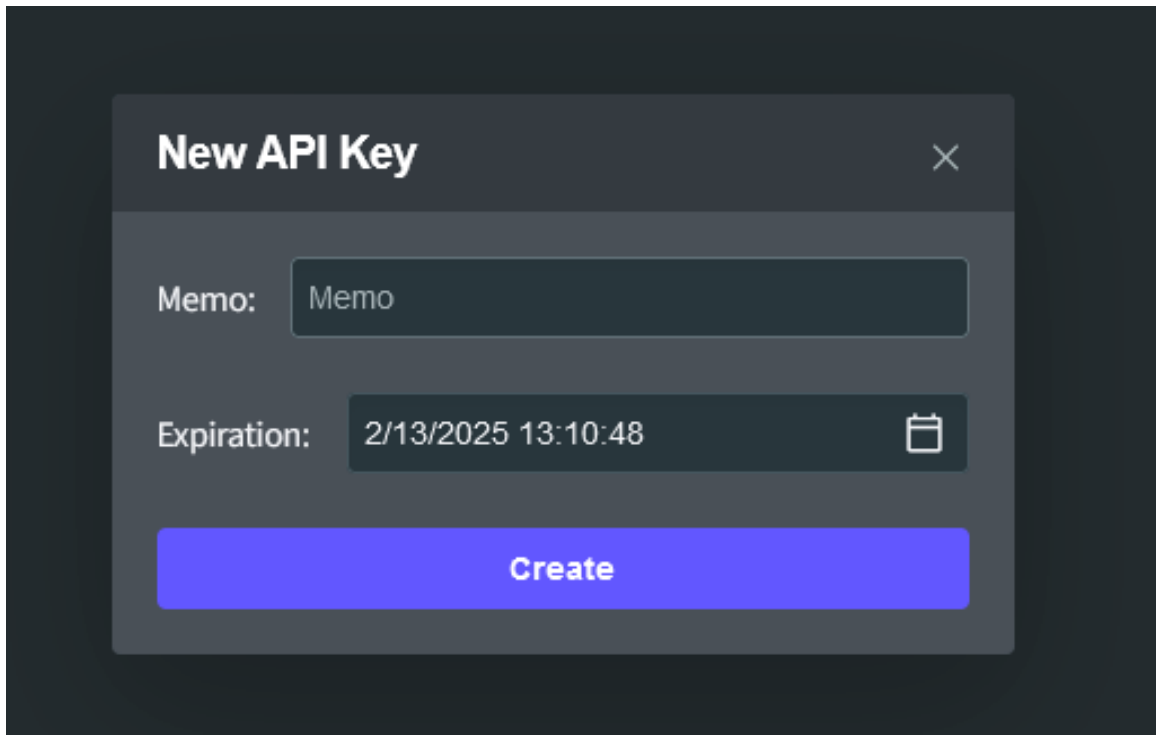


Figure 8: API key management

To add an API key, click on the add button in the upper right-hand corner of the API Keys

---

section on the page. You will be presented with a dialog where you can add a memo to help distinguish the purpose of this API key and set an expiration date.

A dark-themed dialog box titled "New API Key" with a close button (X) in the top right corner. It contains two input fields: "Memo:" with a text box containing the word "Memo", and "Expiration:" with a date/time picker showing "2/13/2025 13:10:48" and a calendar icon. At the bottom is a large blue button labeled "Create".

**New API Key** ✕

Memo:

Expiration:  📅

**Create**

Figure 9: API key creation

After selecting 'Create,' you will be prompted with another dialog to copy the generated API key. You will not be able to retrieve this API key again, so save it somewhere before dismissing the dialog. For information on how to use the API key in a request, see the API section of this document.

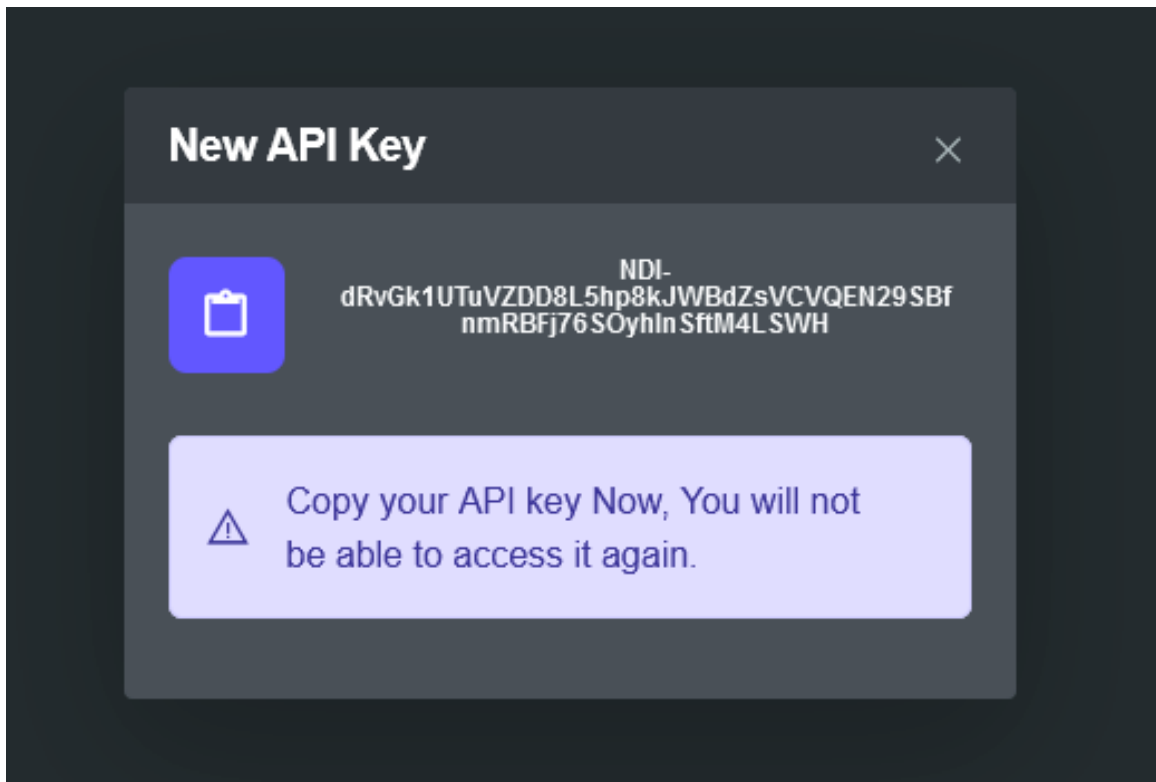


Figure 10: API key copy

### 3.2 Configuration File

NDI Bridge Service uses a JSON configuration file to store settings, similar to the desktop Bridge application. The file is installed under:

`ProgramData\NDI\NDI Bridge Service\service_settings.json`

At the very top of this file are the same five fields that were present on the configuration page of the installer. You can manually modify these settings by following the rules below. Changes to these settings do not take effect until the NDI Bridge Service is restarted, either by restarting the machine or by restarting the NDI Bridge Service manually inside the Windows Service Console. Any invalid configuration, whether syntactic or semantic, will result in the configuration file being reset to its default when the service restarts, or a failure to start the service.

```
"port": 8080,  
"useHttps": false,  
"enableWebContent": true,  
"certificateSubject": "cn=",  
"hostBinding": "127.0.0.1",
```

- 
- *port*: This is the port that the web server uses to provide access to the web interface and APIs. This is not the port used by Bridge in Host or Join mode; that is configured via the API along with all other Bridge settings.
  - *useHttps*: true or false, this field enables or disables SSL. If set to true, you must also provide a valid value for the *certificateSubject* field.
  - *enableWebContent*: true or false, this field determines if the web interface should be accessible. If set to false, only the API will be accessible and no web content will be presented at the URL.
  - *certificateSubject*: If *useHttps* is set to true, you must also provide a valid certificate subject in Common Name format (CN=). SSL certificates must be installed in the local machine certificate store under the logical store path 'Personal/Certificates', as this is the only location NDI Bridge Service looks to find your certificate with a matching CN subject.

To install an SSL certificate on your machine, simply double click the certificate file to open the import wizard. Be sure to select the 'Local Machine' option before proceeding.

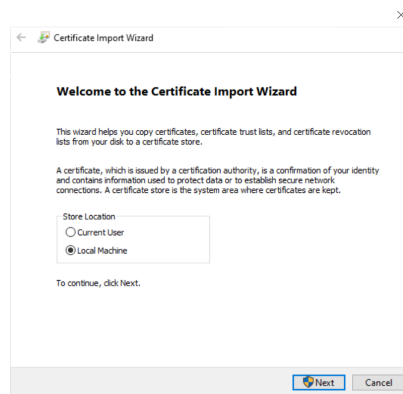


Figure 11: Certificate import

When prompted to select a certificate store, either use the automatic option, or manually select the personal certificate store.

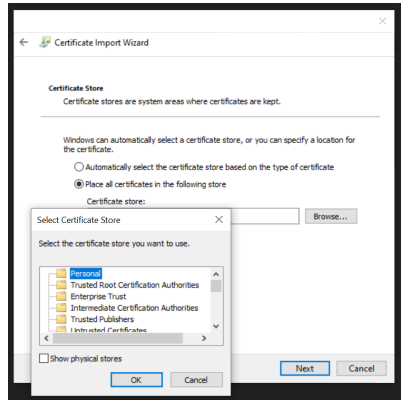


Figure 12: Certificate store

- *hostBinding*: This is the interface that the web server listens on. You can change this to target any particular interface on the machine or use 0.0.0.0 to target all interfaces. If you use 127.0.0.1 then the web interface and API will only be accessible on the local machine.

### 3.3 Manual API Key Management

If you are using SSL and have the web interface disabled, you will have to use the command line to issue and invalidate API keys. Navigate to the install directory and locate *Application.NDI.Bridge.Service.exe*. This application takes the following command line arguments to manage API keys.

*NOTE*: The service must be stopped before modifying API keys, you can stop it in the Windows Service Console.

- *newkey* This argument will prompt you to provide a memo to help identify the purpose of this API key, and the number of days this key will be valid from the current UTC time.
- *listkeys* This argument will provide a list of all API keys that have been issued.
- *removekey* This argument will prompt you to enter in the ID corresponding to the API key you wish to invalidate.

```
Administrator: Windows PowerShell
PS C:\Program Files\NDI\NDI Bridge Service> .\Application.NDI.Bridge.Service.exe newkey
Memo: My Test Key
Days to expire: 90
ID: 66b3dd401cc3be0229aa1cd0
Token: NDI-uzlTeDxiogjsOJE1Be5UfBZIVHnpx8Wty4xwVPIG5QV-dZH1dbXPKBYg3joY
PS C:\Program Files\NDI\NDI Bridge Service> .\Application.NDI.Bridge.Service.exe listkeys
ID: 66b3dd401cc3be0229aa1cd0; Memo: My Test Key; Expiration Date: 2024.11.05 20.46.56 UTC;
PS C:\Program Files\NDI\NDI Bridge Service> .\Application.NDI.Bridge.Service.exe removekey
key ID: 66b3dd401cc3be0229aa1cd0
PS C:\Program Files\NDI\NDI Bridge Service> .\Application.NDI.Bridge.Service.exe listkeys
PS C:\Program Files\NDI\NDI Bridge Service>
```

Figure 13: Command line interface

---

## 4 Webhooks

NDI Bridge Service supports change subscriptions which allow you to observe changes to any value instead of polling. To register a webhook, append the following path to the target API value where the *callback* query parameter is set to the URL you wish to receive POST requests from:

```
/subscribe?callback=https://myurl
```

For instance, if you wanted to subscribe to changes on the host port value you would do:

```
api/host/port/subscribe?callback=https://myurl
```

If a subscription is successful, you will receive a GUID as a response. This GUID is the handle for your subscription. If you want to unsubscribe from any change subscription you call the following URL (do not append to the target API path) with the *id* query parameter being set to the GUID you received from the subscribe response:

```
http://localhost:8080/api/unsubscribe?id=
```

Change subscriptions are POSTed to the callback URL as JSON with *key* being the target API path (like *host/port*) and *value* being the current value or "Value changed." for security sensitive values.

Active webhooks can be seen under the 'Webhooks' section of the web interface.

Webhooks

Bridge Service supports change subscriptions which allow you to observe changes to any value instead of polling for value changes.

Webhooks

To register a Webhook you simply have to append the following path to the target API value where the `callback` query parameter is set to the URL you wish to receive POST requests from: `/subscribe?callback=https://myurl`

If a subscription is successful, you will receive a GUID as a response. This GUID is the handle for your subscription. If you want to unsubscribe from any change subscription you call the following URL (not appended to the target API path) with the `id` query parameter being set to the GUID you received from the subscribe response: `http://localhost:8080/api/unsubscribe?id=`

For instance, if you wanted to subscribe to changes on the host port value you would do: `api/host/port/subscribe?callback=https://myurl`

Change subscriptions are POSTed to the callback URL as JSON with `key` being the target API path (like `host/port`) and `value` being the current value or "Value changed." for security sensitive values.

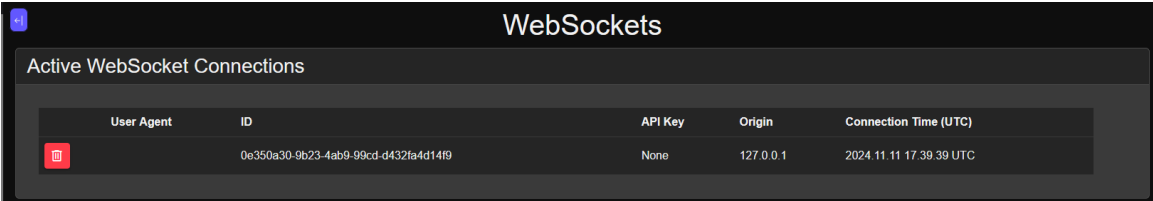
Active Webhooks

| ID                                                                                                                      | Status | User Agent                                                                       | API Key | Path   | Destination                                               | IP        | Date Created (UTC)      |
|-------------------------------------------------------------------------------------------------------------------------|--------|----------------------------------------------------------------------------------|---------|--------|-----------------------------------------------------------|-----------|-------------------------|
|  c9a1bc12-1c89-4714-8550-1b3dd445ca9 | Active | Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0 | None    | uptime | https://webhook.site/4ccb2856-017e-4978-b1f8-1fd970a6fdb8 | 127.0.0.1 | 2024.11.11 17:37:28 UTC |

Figure 14: Command line interface

## 5 WebSockets

Bridge Service supports WebSocket connections for listening to changes in any property. Logs are also sent over WebSocket connections. Data transmitted through the WebSocket connection is in JSON format. Active WebSocket connections can be viewed under the 'WebSockets' section of the web interface.



| Active WebSocket Connections |                                      |         |           |                         |
|------------------------------|--------------------------------------|---------|-----------|-------------------------|
| User Agent                   | ID                                   | API Key | Origin    | Connection Time (UTC)   |
|                              | 0e350a30-9b23-4ab9-99cd-d432fa4d14f9 | None    | 127.0.0.1 | 2024.11.11 17:39:39 UTC |

Figure 15: Active WebSockets

```

    ], "EventId": {"Id": 1, "Name": "NDI Discovery Service"}, "Facility": "NDI Discovery Service", "LogLevel": 2, "EventName": "n
all, "Timestamp": "2024-11-11T17:40:11.2139866Z", "Exception": null}
    "Key": "uptime", "Value": "00:02:51.0052886"}
    "Key": "uptime", "Value": "00:02:52.0048536"}
    "Key": "uptime", "Value": "00:02:53.0055876"}
    "Key": "uptime", "Value": "00:02:54.0045610"}
    "Key": "uptime", "Value": "00:02:55.0046378"}
    "Type": 1, "Value": null, "Message": "[NDI Discovery Service]    0:02:55 : Listening... [--*
    ], "EventId": {"Id": 1, "Name": "NDI Discovery Service"}, "Facility": "NDI Discovery Service", "LogLevel": 2, "EventName": "n
all, "Timestamp": "2024-11-11T17:40:16.226365Z", "Exception": null}
    "Key": "uptime", "Value": "00:02:56.0046900"}
    "Key": "uptime", "Value": "00:02:57.0049331"}
    "Key": "uptime", "Value": "00:02:58.0055740"}
    "Key": "uptime", "Value": "00:02:59.0048589"}
    "Key": "uptime", "Value": "00:03:00.0046039"}
    "Type": 1, "Value": null, "Message": "[NDI Discovery Service]    0:03:00 : Listening... [---*
    ], "EventId": {"Id": 1, "Name": "NDI Discovery Service"}, "Facility": "NDI Discovery Service", "LogLevel": 2, "EventName": "n
all, "Timestamp": "2024-11-11T17:40:21.2386509Z", "Exception": null}
    "Type": 0, "Value": null, "Message": "Bridge service running: 11/11/2024 17:40:21 \u002B00:00", "EventId": {"Id": 0, "Name":
null}, "Facility": null, "LogLevel": 2, "EventName": null, "Timestamp": "2024-11-11T17:40:21.4188569Z", "Exception": null}
    "Key": "uptime", "Value": "00:03:01.0054654"}
    "Key": "uptime", "Value": "00:03:02.0047482"}
    "Key": "uptime", "Value": "00:03:03.0046385"}
    "Key": "uptime", "Value": "00:03:04.0044756"}
    "Key": "uptime", "Value": "00:03:05.0047647"}
    "Type": 1, "Value": null, "Message": "[NDI Discovery Service]    0:03:05 : Listening... [----*
    ], "EventId": {"Id": 1, "Name": "NDI Discovery Service"}, "Facility": "NDI Discovery Service", "LogLevel": 2, "EventName": "n
all, "Timestamp": "2024-11-11T17:40:26.2521563Z", "Exception": null}

```

Figure 16: Example WebSocket Output

---

The C# code bellow creates a WebSocket client, establishes a connection to a WebSocket server and listens for incoming messages:

```
using var client = new ClientWebSocket();

// The websocket path is /stream
// When using SSL there's also need to use the WSS protocol and
// Supply an API key in the x-api-key request header
var endpoint = "ws://127.0.0.1:8080/stream";

// var endpoint = "wss://127.0.0.1:8080/stream"; // for ssl
// client.Options.SetRequestHeader("x-api-key", /* you key */); // for ssl

// Optionally set a user agent
client.Options.SetRequestHeader("User-Agent", "My Connection");

await client.ConnectAsync(new Uri(endpoint), CancellationToken.None);
Console.WriteLine("Connected to WebSocket server.");

var buffer = new byte[4096];
while (client.State == WebSocketState.Open)
{
    var result = await client.ReceiveAsync(new ArraySegment<byte>(buffer),
        CancellationToken.None);

    if (result.MessageType == WebSocketMessageType.Text)
    {
        string message = Encoding.UTF8.GetString(buffer, 0, result.Count);
        Console.WriteLine(message);
    }
    else if (result.MessageType == WebSocketMessageType.Close)
    {
        await client.CloseAsync(WebSocketCloseStatus.NormalClosure, "Closing",
            CancellationToken.None);
        Console.WriteLine("Connection closed.");
    }
}
```

# 6 API

All API calls use GET and or POST requests to retrieve and set values. Every response from a GET or POST request use the same structure. The response is in JSON format and includes a true / false field to indicate success, an error field to provide more detailed information if there is an error, and a content field that contains either the current value of the setting or a message detailing what action the request performed.

```
{"success":true,"error":null,"content":0}
```

In addition to this document, the web interface provides a full list of API calls and their use cases.

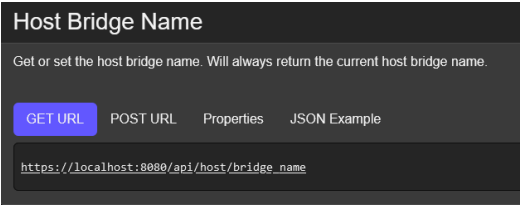


Figure 17: API GET URL

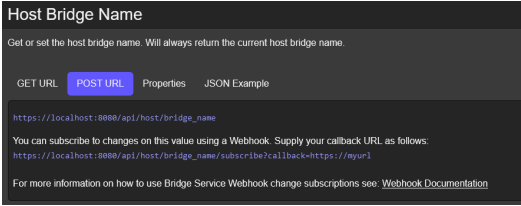


Figure 18: API POST URL and Instructions

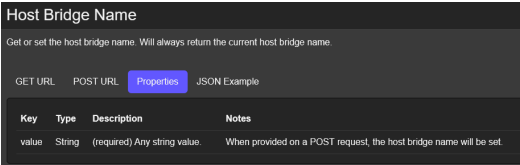


Figure 19: API Properties

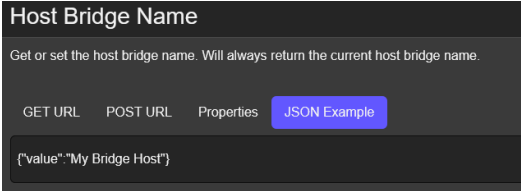


Figure 20: API JSON Response

---

## 6.1 API Calls Using SSL

When SSL is enabled you must include a valid API key in the header of your request. It key in the header should be named 'x-api-key' and the value should be the API key you issued starting with 'NDI-'. Here is some example C# code for sending GET and POST requests with SSL where 'API\_KEY' is a string value containing the 'NDI-' prefixed key:

```
static async Task<string> PostAsync(string uri, Dictionary<string, string> values)
{
    var client = new HttpClient();
    client.DefaultRequestHeaders.Add("x-api-key", API_KEY);

    var content = JsonConvert.Create(values);
    var result = await client.PostAsync(uri, content);
    return await result.Content.ReadAsStringAsync();
}
```

```
static async Task<string> GetAsync(string uri)
{
    var client = new HttpClient();
    client.DefaultRequestHeaders.Add("x-api-key", API_KEY);

    var result = await client.GetAsync(uri);
    return await result.Content.ReadAsStringAsync();
}
```

---

The following subsections detail each API endpoint available in NDI Bridge Service.

## 6.2 Auto Start

Get or set whether to start join or host on service start-up. Will always return the latest auto start value. The service must be restarted to reflect this change.

| Key   | Type   | Description                                                            | Notes                                                                          |
|-------|--------|------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| value | String | (required) 0, 1 or 2 where 0 = None, 1 = Host, 2 = Join, and 3 = Local | If provided on a POST request, the Bridge Service auto start mode will be set. |

GET URL: `http://localhost:8080/api/auto_start`  
POST URL: `http://localhost:8080/api/auto_start`

EXAMPLE POST CONTENT: `{"value":"0"}`

## 6.3 GPU List

Gets the list of available GPUs.

GET URL: `http://localhost:8080/api/gpu_list`

## 6.4 Host Bandwidth

Get the current host bandwidth message.

GET URL: `http://localhost:8080/api/host/bandwidth_message`

## 6.5 Host Bridge Name

Get or set the host bridge name. Will always return the latest host bridge name.

| Key   | Type   | Description                  | Notes                                                            |
|-------|--------|------------------------------|------------------------------------------------------------------|
| value | String | (required) Any string value. | If provided on a POST request, the host bridge name will be set. |

---

GET URL: `http://localhost:8080/api/host/bridge_name`  
POST URL: `http://localhost:8080/api/host/bridge_name`

EXAMPLE POST CONTENT: `{"value":"My Bridge Host"}`

## 6.6 Host Buffer

Get or set the host buffer value. Will always return the latest host buffer value.

| Key   | Type   | Description                                   | Notes                                                                                                                |
|-------|--------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| value | String | (optional) A buffer length value from 0-3000. | Buffer length value in the range of 0-3000. If provided on a POST request, the host buffer length value will be set. |

GET URL: `http://localhost:8080/api/host/buffer`  
POST URL: `http://localhost:8080/api/host/buffer`

EXAMPLE POST CONTENT: `{"value":"0"}`

## 6.7 Host Clear Encryption Key

Clears the host encryption key on POST.

POST URL: `http://localhost:8080/api/host/clear_encryption_key`

EXAMPLE POST CONTENT: `{}`

## 6.8 Host GPU Limit

Gets a boolean to represent if the host has reached its GPU encoding limits or not.

GET URL: `http://localhost:8080/api/host/gpu_limit`

---

## 6.9 Host Groups

Get or set the host groups to share. Will always return the latest host groups.

| Key   | Type   | Description                                        | Notes                                                       |
|-------|--------|----------------------------------------------------|-------------------------------------------------------------|
| value | String | (required) A comma delineated list of group names. | If provided on a POST request, the host groups will be set. |

GET URL: `http://localhost:8080/api/host/groups`

POST URL: `http://localhost:8080/api/host/groups`

EXAMPLE POST CONTENT: `{"value":"public,test group"}`

## 6.10 Host HX Encoder

Get or set the host HX encoder. Will always return the latest host HX encoder value.

| Key   | Type   | Description                                         | Notes                                                                |
|-------|--------|-----------------------------------------------------|----------------------------------------------------------------------|
| value | String | (required) 0 or 1 where 0 = h.264, 1 = HEVC (h.265) | If provided on a POST request, the host HX encoder type will be set. |

GET URL: `http://localhost:8080/api/host/hx_encoder`

POST URL: `http://localhost:8080/api/host/hx_encoder`

EXAMPLE POST CONTENT: `{"value":"0"}`

## 6.11 Host HX GPU

Get or set the host HX GPU. Will always return the latest host HX GPU.

| Key   | Type   | Description                                                               | Notes                                                       |
|-------|--------|---------------------------------------------------------------------------|-------------------------------------------------------------|
| value | String | (required) The name of the GPU. Use 'Auto' to automatically select a GPU. | If provided on a POST request, the host HX GPU will be set. |

GET URL: `http://localhost:8080/api/host/hx_gpu`

POST URL: `http://localhost:8080/api/host/hx_gpu`

EXAMPLE POST CONTENT: `{"value":"Auto"}`

---

## 6.12 Host HX NDI4 Compatibility Mode

Get or set the host HX NDI4 compatibility mode. Will always return the latest host HX NDI4 compatibility mode value.

| Key   | Type   | Description                                                        | Notes                                                                           |
|-------|--------|--------------------------------------------------------------------|---------------------------------------------------------------------------------|
| value | String | (required) A boolean to enable or disable NDI4 compatibility mode. | If provided on a POST request, the host HX NDI4 compatibility mode will be set. |

GET URL: `http://localhost:8080/api/host/hx_ndi4_compatibility_mode`

POST URL: `http://localhost:8080/api/host/hx_ndi4_compatibility_mode`

EXAMPLE POST CONTENT: `{"value":"false"}`

## 6.13 Host HX Output

Get or set the host HX output. Will always return the latest host HX output value.

| Key   | Type   | Description                                                                                | Notes                                                               |
|-------|--------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| value | String | (required) 0, 1, or 2 where 0 = NDI — HX, 1 = NDI High Bandwidth, and 2 = Do Not Transcode | If provided on a POST request, the host HX output type will be set. |

GET URL: `http://localhost:8080/api/host/hx_output`

POST URL: `http://localhost:8080/api/host/hx_output`

EXAMPLE POST CONTENT: `{"value":"0"}`

## 6.14 Host HX Quality

Get or set the host HX quality setting. Will always return the latest host HX quality setting value.

| Key   | Type   | Description                                                             | Notes                                                                   |
|-------|--------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| value | String | (required) A value between 10 and 400 with 10 being the lowest quality. | If provided on a POST request, the host HX quality setting will be set. |

GET URL: `http://localhost:8080/api/host/hx_quality`

POST URL: `http://localhost:8080/api/host/hx_quality`

---

EXAMPLE POST CONTENT: {"value":"100"}

## 6.15 Host Port

Get or set the host port value. Will always return the latest host port value.

| Key   | Type   | Description                                     | Notes                                                                                                |
|-------|--------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| value | String | (required) Port number in the range of 1-65545. | Port number in the range of 1-65545. If provided on a POST request, the host port value will be set. |

GET URL: `http://localhost:8080/api/host/port`  
POST URL: `http://localhost:8080/api/host/port`

EXAMPLE POST CONTENT: {"value":"5990"}

## 6.16 Host Set Encryption Key

Sets the host's encryption key on POST.

| Key   | Type   | Description                                 | Notes                                                                                            |
|-------|--------|---------------------------------------------|--------------------------------------------------------------------------------------------------|
| value | String | (required) The encryption key string value. | Returns success on setting the encryption key. Does not return the current encryption key value. |

POST URL: `http://localhost:8080/api/host/set_encryption_key`

EXAMPLE POST CONTENT: {"value":"some encryption key"}

## 6.17 Host Start

Starts the host and returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/host/start`

EXAMPLE POST CONTENT: {"value":"Host started."}

---

## 6.18 Host Status

Gets the current host status message.

GET URL: `http://localhost:8080/api/host/status_message`

## 6.19 Host Stop

Stops the host if the host was running. Returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/host/stop`

EXAMPLE POST CONTENT: `{"value":"Host stopped."}`

## 6.20 Is Running

Returns true if Bridge is running in Host or Join mode.

GET URL: `http://localhost:8080/api/is_running`

## 6.21 Join Bandwidth

Get the current join bandwidth message.

GET URL: `http://localhost:8080/api/join/bandwidth_message`

## 6.22 Join Bridge Name

Get or set the join bridge name. Will always return the latest join bridge name.

| Key   | Type   | Description                  | Notes                                                            |
|-------|--------|------------------------------|------------------------------------------------------------------|
| value | String | (required) Any string value. | If provided on a POST request, the join bridge name will be set. |

GET URL: `http://localhost:8080/api/join/bridge_name`

POST URL: `http://localhost:8080/api/join/bridge_name`

EXAMPLE POST CONTENT: `{"value":"My Bridge Name"}`

---

## 6.23 Join Buffer

Get or set the join buffer value. Will always return the latest join buffer value.

| Key   | Type   | Description                                   | Notes                                                                                                                |
|-------|--------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| value | String | (required) A buffer length value from 0-3000. | Buffer length value in the range of 0-3000. If provided on a POST request, the join buffer length value will be set. |

GET URL: `http://localhost:8080/api/join/buffer`

POST URL: `http://localhost:8080/api/join/buffer`

EXAMPLE POST CONTENT: `{"value":"100"}`

## 6.24 Join Clear Encryption Key

Clears the join encryption key when called.

POST URL: `http://localhost:8080/api/join/clear_encryption_key`

EXAMPLE POST CONTENT: `{}`

## 6.25 Join GPU Limit

Gets a boolean to represent if the join has reached its GPU encoding limits or not.

GET URL: `http://localhost:8080/api/join/gpu_limit`

## 6.26 Join Groups

Get or set the join groups to share. Will always return the latest join groups.

| Key   | Type   | Description                                        | Notes                                                       |
|-------|--------|----------------------------------------------------|-------------------------------------------------------------|
| value | String | (required) A comma delineated list of group names. | If provided on a POST request, the join groups will be set. |

---

GET URL: `http://localhost:8080/api/join/groups`  
POST URL: `http://localhost:8080/api/join/groups`

EXAMPLE POST CONTENT: `{"value":"public,test"}`

## 6.27 Join HX Encoder

Get or set the join HX encoder. Will always return the latest join HX encoder value.

| Key   | Type   | Description                                         | Notes                                                                |
|-------|--------|-----------------------------------------------------|----------------------------------------------------------------------|
| value | String | (required) 0 or 1 where 0 = h.264, 1 = HEVC (h.265) | If provided on a POST request, the join HX encoder type will be set. |

GET URL: `http://localhost:8080/api/join/hx_encoder`  
POST URL: `http://localhost:8080/api/join/hx_encoder`

EXAMPLE POST CONTENT: `{"value":"0"}`

## 6.28 Join HX GPU

Get or set the join HX GPU. Will always return the latest join HX GPU.

| Key   | Type   | Description                                                               | Notes                                                       |
|-------|--------|---------------------------------------------------------------------------|-------------------------------------------------------------|
| value | String | (required) The name of the GPU. Use 'Auto' to automatically select a GPU. | If provided on a POST request, the join HX GPU will be set. |

GET URL: `http://localhost:8080/api/join/hx_gpu`  
POST URL: `http://localhost:8080/api/join/hx_gpu`

EXAMPLE POST CONTENT: `{"value":"Auto"}`

## 6.29 Join HX NDI4 Compatibility Mode

Get or set the join HX NDI4 compatibility mode. Will always return the latest join HX NDI4 compatibility mode value.

---

| Key   | Type   | Description                                                        | Notes                                                                           |
|-------|--------|--------------------------------------------------------------------|---------------------------------------------------------------------------------|
| value | String | (required) A boolean to enable or disable NDI4 compatibility mode. | If provided on a POST request, the join HX NDI4 compatibility mode will be set. |

GET URL: `http://localhost:8080/api/join/hx_ndi4_compatibility_mode`  
 POST URL: `http://localhost:8080/api/join/hx_ndi4_compatibility_mode`

EXAMPLE POST CONTENT: `{"value":"false"}`

### 6.30 Join HX Output

Get or set the join HX output. Will always return the latest join HX output value.

| Key   | Type   | Description                                                                                | Notes                                                               |
|-------|--------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| value | String | (required) 0, 1, or 2 where 0 = NDI — HX, 1 = NDI High Bandwidth, and 2 = Do Not Transcode | If provided on a POST request, the join HX output type will be set. |

GET URL: `http://localhost:8080/api/join/hx_output`  
 POST URL: `http://localhost:8080/api/join/hx_output`

EXAMPLE POST CONTENT: `{"value":"0"}`

### 6.31 Join HX Quality

Get or set the join HX quality setting. Will always return the latest join HX quality setting value.

| Key   | Type   | Description                                                             | Notes                                                                   |
|-------|--------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| value | String | (required) A value between 10 and 400 with 10 being the lowest quality. | If provided on a POST request, the join HX quality setting will be set. |

GET URL: `http://localhost:8080/api/join/hx_quality`  
 POST URL: `http://localhost:8080/api/join/hx_quality`

EXAMPLE POST CONTENT: `{"value":"100"}`

---

### 6.32 Join IP Address

Get or set the join IP address. Will always return the latest join IP address.

| Key   | Type   | Description                                      | Notes                                                           |
|-------|--------|--------------------------------------------------|-----------------------------------------------------------------|
| value | String | (required) IP Address in IPv4 format '127.0.0.1' | If provided on a POST request, the join IP address will be set. |

GET URL: `http://localhost:8080/api/join/ip_address`

POST URL: `http://localhost:8080/api/join/ip_address`

EXAMPLE POST CONTENT: `{"value":"127.0.0.1"}`

### 6.33 Join Port

Get or set the join port value. Will always return the latest join port value.

| Key   | Type   | Description                                     | Notes                                                                                                |
|-------|--------|-------------------------------------------------|------------------------------------------------------------------------------------------------------|
| value | String | (required) Port number in the range of 1-65535. | Port number in the range of 1-65535. If provided on a POST request, the join port value will be set. |

GET URL: `http://localhost:8080/api/join/port`

POST URL: `http://localhost:8080/api/join/port`

EXAMPLE POST CONTENT: `{"value":"5990"}`

### 6.34 Join Set Encryption Key

Sets the join encryption key on POST.

| Key   | Type   | Description                                 | Notes                                                                                            |
|-------|--------|---------------------------------------------|--------------------------------------------------------------------------------------------------|
| value | String | (required) The encryption key string value. | Returns success on setting the encryption key. Does not return the current encryption key value. |

---

POST URL: `http://localhost:8080/api/join/set_encryption_key`

EXAMPLE POST CONTENT: `{"value":"some encryption key"}`

### 6.35 Join Start

Starts the join and returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/join/start`

EXAMPLE POST CONTENT: `{"value":"Join started."}`

### 6.36 Join Status

Gets the current join status message.

GET URL: `http://localhost:8080/api/join/status_message`

### 6.37 Join Stop

Stops the join if the join was running. Returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/join/stop`

EXAMPLE POST CONTENT: `{"value":"Join stopped."}`

### 6.38 Local Bridge Name

Get or set the local bridge name. Will always return the latest local bridge name.

| Key   | Type   | Description                  | Notes                                                             |
|-------|--------|------------------------------|-------------------------------------------------------------------|
| value | String | (required) Any string value. | If provided on a POST request, the local bridge name will be set. |

GET URL: `http://localhost:8080/api/local/bridge_name`

POST URL: `http://localhost:8080/api/local/bridge_name`

EXAMPLE POST CONTENT: `{"value":"My Bridge Name"}`

---

## 6.39 Local Groups

Get or set the local groups to share. Will always return the latest local groups.

| Key   | Type   | Description                                        | Notes                                                        |
|-------|--------|----------------------------------------------------|--------------------------------------------------------------|
| value | String | (required) A comma delineated list of group names. | If provided on a POST request, the local groups will be set. |

GET URL: <http://localhost:8080/api/local/groups>

POST URL: <http://localhost:8080/api/local/groups>

EXAMPLE POST CONTENT: {"value":"public,test"}

## 6.40 Local HX GPU

Get or set the local HX GPU. Will always return the latest local HX GPU.

| Key   | Type   | Description                                                               | Notes                                                        |
|-------|--------|---------------------------------------------------------------------------|--------------------------------------------------------------|
| value | String | (required) The name of the GPU. Use 'Auto' to automatically select a GPU. | If provided on a POST request, the local HX GPU will be set. |

GET URL: [http://localhost:8080/api/local/hx\\_gpu](http://localhost:8080/api/local/hx_gpu)

POST URL: [http://localhost:8080/api/local/hx\\_gpu](http://localhost:8080/api/local/hx_gpu)

EXAMPLE POST CONTENT: {"value":"Auto"}

## 6.41 Local HX NDI4 Compatibility Mode

Get or set the local HX NDI4 compatibility mode. Will always return the latest local HX NDI4 compatibility mode value.

| Key   | Type   | Description                                                        | Notes                                                                            |
|-------|--------|--------------------------------------------------------------------|----------------------------------------------------------------------------------|
| value | String | (required) A boolean to enable or disable NDI4 compatibility mode. | If provided on a POST request, the local HX NDI4 compatibility mode will be set. |

GET URL: [http://localhost:8080/api/local/hx\\_ndi4\\_compatibility\\_mode](http://localhost:8080/api/local/hx_ndi4_compatibility_mode)

POST URL: [http://localhost:8080/api/local/hx\\_ndi4\\_compatibility\\_mode](http://localhost:8080/api/local/hx_ndi4_compatibility_mode)

---

EXAMPLE POST CONTENT: {"value":"false"}

## 6.42 Local HX Output

Get or set the local HX output. Will always return the latest local HX output value.

| Key   | Type   | Description                                                                                | Notes                                                                |
|-------|--------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| value | String | (required) 0, 1, or 2 where 0 = NDI — HX, 1 = NDI High Bandwidth, and 2 = Do Not Transcode | If provided on a POST request, the local HX output type will be set. |

GET URL: [http://localhost:8080/api/local/hx\\_output](http://localhost:8080/api/local/hx_output)

POST URL: [http://localhost:8080/api/local/hx\\_output](http://localhost:8080/api/local/hx_output)

EXAMPLE POST CONTENT: {"value":"0"}

## 6.43 Local HX Quality

Get or set the local HX quality setting. Will always return the latest local HX quality setting value.

| Key   | Type   | Description                                                             | Notes                                                                    |
|-------|--------|-------------------------------------------------------------------------|--------------------------------------------------------------------------|
| value | String | (required) A value between 10 and 400 with 10 being the lowest quality. | If provided on a POST request, the local HX quality setting will be set. |

GET URL: [http://localhost:8080/api/local/hx\\_quality](http://localhost:8080/api/local/hx_quality)

POST URL: [http://localhost:8080/api/local/hx\\_quality](http://localhost:8080/api/local/hx_quality)

EXAMPLE POST CONTENT: {"value":"100"}

## 6.44 Local Send Groups

Get or set the send NDI Groups where locally the sources from this Bridge will appear in. Will always return the current local send groups.

| Key   | Type   | Description                                              | Notes                                                             |
|-------|--------|----------------------------------------------------------|-------------------------------------------------------------------|
| value | String | (required) A comma delineated list of send groups names. | If provided on a POST request, the local send groups will be set. |

---

GET URL: `http://localhost:8080/api/local/send_groups`  
POST URL: `http://localhost:8080/api/local/send_groups`

EXAMPLE POST CONTENT: `{"value":"public,test group"}`

## 6.45 Local Start

Starts the local bridge and returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/local/start`

EXAMPLE POST CONTENT: `{"value":"Local started."}`

## 6.46 Local Status

Gets the current local status message.

GET URL: `http://localhost:8080/api/local/status_message`

## 6.47 Local Stop

Stops the local if the local was running. Returns if the operation succeeded or not.

POST URL: `http://localhost:8080/api/local/stop`

EXAMPLE POST CONTENT: `{"value":"Local stopped."}`

## 6.48 Connection Test Start

Starts the connection test. You can use the connection test to help identify the best buffer setting for your chosen bitrate given your current network conditions. You must have at least one active connection to perform a connection test.

POST URL: `http://localhost:8080/api/connection_test_start`

---

## 6.49 Connection Test Stop

Stops the connection test. The response to this request will include the recommended buffer setting.

POST URL: `http://localhost:8080/api/connection_test_stop`

## 6.50 Connection Test Bitrate

Get or set the bitrate used in the connection test. Will always return the current bitrate value.

| Key   | Type   | Description                                                                                                                                                                 | Notes |
|-------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| value | String | (required) Value between 1 and the maximum speed reported by the network adapters. When provided on a POST request, the Bridge Service connection test bitrate will be set. |       |

GET URL: `http://localhost:8080/api/connection_test_bitrate`

POST URL: `http://localhost:8080/api/connection_test_bitrate`

## 6.51 Run Mode

Gets the current running mode.

GET URL: `http://localhost:8080/api/runmode`

## 6.52 Uptime

Returns the timespan since the service started.

GET URL: `http://localhost:8080/api/uptime`